

explain me datetime module in detailed and examples

Absolutely. Since you're learning Python for **data engineering and Spark Structured Streaming**, `datetime` is especially important for timestamps, event time, file names, scheduling, and time-based data processing.

Python `datetime` Module

The `datetime` module is Python's standard library for working with:

- Dates
- Times
- Date + time together
- Time differences
- Time zones
- Formatting and parsing dates
- Adding/subtracting time

You import it with:

```
</> Python
```



▶ Run

```
import datetime
```

1. The main objects in `datetime`

The most important classes are:

```
datetime
├── date
├── time
├── datetime
└── timedelta
```



There is also:

```
timezone
```



Think about them like this:

Object	Represents	Example
date	Date only	2026-08-29
time	Time only	14:30:45
datetime	Date + time	2026-08-29 14:30:45
timedelta	Duration/difference	2 days, 3 hours
timezone	Time-zone information	UTC, +03:00

2. date

If you only need the date:

</> Python

from datetime import date

today = date.today()

print(today)

Run

Output:

2026-08-29

You can access individual parts:

</> Python

from datetime import date

today = date.today()

print(today.year)

print(today.month)

print(today.day)

Run

Output:

```
2026
8
29
```



3. Creating a specific date

You can create a date manually:

```
</> Python

from datetime import date

my_date = date(2026, 8, 29)

print(my_date)
```



▶ Run

Output:

```
2026-08-29
```



The order is:

```
</> Python

date(year, month, day)
```



▶ Run

For example:

```
</> Python

birthday = date(2001, 11, 4)

print(birthday)
```



▶ Run

4. time

`time` represents a time without a date.

```
</> Python
```



▶ Run

```
from datetime import time

t = time(14, 30, 45)

print(t)
```

Output:

```
14:30:45
```



The structure is:

```
</> Python

time(hour, minute, second)
```



▶ Run

You can also include microseconds:

```
</> Python

t = time(14, 30, 45, 123456)

print(t)
```



▶ Run

Output:

```
14:30:45.123456
```



Access individual values:

```
</> Python

print(t.hour)
print(t.minute)
print(t.second)
print(t.microsecond)
```



▶ Run

5. datetime

This is probably the **most important class** for data engineering.

It contains:

```
date + time
```



Example:

</> Python



▶ Run

```
from datetime import datetime

now = datetime.now()

print(now)
```

Example output:

2026-08-29 20:19:32.452781



It contains:

year
month
day
hour
minute
second
microsecond



6. Getting individual components

</> Python



▶ Run

```
from datetime import datetime

now = datetime.now()

print("Year:", now.year)
print("Month:", now.month)
print("Day:", now.day)

print("Hour:", now.hour)
print("Minute:", now.minute)
print("Second:", now.second)
```

Example:

```
Year: 2026
Month: 8
Day: 29
Hour: 20
Minute: 19
Second: 32
```



This is useful when generating files.

For example:

```
</> Python
```



▶ Run

```
from datetime import datetime

now = datetime.now()

filename = f"orders_{now.year}_{now.month}_{now.day}.json"

print(filename)
```

Output:

```
orders_2026_8_29.json
```



7. Better file names with `strftime()`

Instead of manually extracting every component, use:

```
</> Python
```



▶ Run

```
strftime()
```

Example:

```
</> Python
```



▶ Run

```
from datetime import datetime

now = datetime.now()

filename = now.strftime("%Y-%m-%d")

print(filename)
```

Output:

2026-08-29



You can create:

</> Python



▶ Run

```
filename = now.strftime("orders_%Y-%m-%d.json")

print(filename)
```

Output:

orders_2026-08-29.json



This is extremely useful for your **JSON file generation practice**.

8. Important strftime() codes

These are worth memorizing:

Code	Meaning	Example
%Y	4-digit year	2026
%y	2-digit year	26
%m	Month	08
%d	Day	29
%H	Hour, 24-hour	20
%I	Hour, 12-hour	08
%M	Minute	19
%S	Second	32
%f	Microsecond	452781
%A	Full weekday	Saturday

Code	Meaning	Example
%a	Short weekday	Sat
%B	Full month	August
%b	Short month	Aug

Example:

</> Python



▶ Run

```
from datetime import datetime

now = datetime.now()

print(now.strftime("%Y"))
print(now.strftime("%m"))
print(now.strftime("%d"))
print(now.strftime("%H"))
```

9. Combining them

You can create a nice timestamp:

</> Python



▶ Run

```
from datetime import datetime

now = datetime.now()

timestamp = now.strftime("%Y-%m-%d %H:%M:%S")

print(timestamp)
```

Output:

2026-08-29 20:19:32



This format is extremely common in data engineering.

For example, JSON:

</> JSON



```
{
  "order_id": 101,
  "price": 50,
  "quantity": 2,
  "event_time": "2026-08-29 20:19:32"
}
```

10. strptime() — String → datetime

This is the opposite of `strftime()`.

`strftime`

datetime → string



`strptime`

string → datetime



Example:

</> Python



Run

```
from datetime import datetime

date_string = "2026-08-29 20:19:32"

dt = datetime.strptime(date_string, "%Y-%m-%d %H:%M:%S")

print(dt)
print(type(dt))
```

Output:

```
2026-08-29 20:19:32
<class 'datetime.datetime'>
```



This is important when reading timestamps from:

- JSON
- CSV

- databases
- APIs
- Kafka messages
- log files

11. Another `strptime()` example

Suppose your JSON contains:

```
29/08/2026 20:30:00
```



You can convert it:

```
</> Python
```



▶ Run

```
from datetime import datetime

text = "29/08/2026 20:30:00"

dt = datetime.strptime(text, "%d/%m/%Y %H:%M:%S")

print(dt)
```

Output:

```
2026-08-29 20:30:00
```



12. `timedelta`

`timedelta` represents a **duration**.

For example:

```
5 seconds
3 minutes
2 hours
7 days
```



Import it:

```
</> Python
```



▶ Run

```
from datetime import timedelta
```

Create one:

</> Python



▶ Run

```
duration = timedelta(days=2)

print(duration)
```

Output:

2 days, 0:00:00



13. Adding days

This is very useful.

</> Python



▶ Run

```
from datetime import datetime, timedelta

now = datetime.now()

future = now + timedelta(days=7)

print("Now:", now)
print("Future:", future)
```

You can also subtract:

</> Python



▶ Run

```
past = now - timedelta(days=7)

print(past)
```

14. Adding hours

</> Python



▶ Run

```
from datetime import datetime, timedelta

now = datetime.now()

future = now + timedelta(hours=5)

print(future)
```

15. Adding minutes

</> Python



▶ Run

```
future = now + timedelta(minutes=30)
```

Seconds:

</> Python



▶ Run

```
future = now + timedelta(seconds=45)
```

You can combine them:

</> Python



▶ Run

```
future = now + timedelta(
    days=2,
    hours=5,
    minutes=30
)

print(future)
```

16. Difference between two dates

This is one of the most useful things about `datetime` .

</> Python



▶ Run

```
from datetime import date

start = date(2026, 8, 1)
end = date(2026, 8, 29)

difference = end - start

print(difference)
```

Output:

28 days, 0:00:00



You can get the number of days:

```
</> Python

print(difference.days)
```



▶ Run

Output:

28



17. Difference between two datetimes

```
</> Python

from datetime import datetime

start = datetime(2026, 8, 29, 10, 0, 0)
end = datetime(2026, 8, 29, 15, 30, 0)

difference = end - start

print(difference)
```



▶ Run

Output:

5:30:00



You can get:

```
</> Python
```



▶ Run

```
print(difference.total_seconds())
```

Output:

```
19800.0
```

Because:

```
5.5 hours × 60 × 60 = 19,800 seconds
```

18. Comparing dates

You can compare dates directly.

</> Python



▶ Run

```
from datetime import date

date1 = date(2026, 8, 29)
date2 = date(2026, 9, 1)

print(date1 < date2)
print(date1 > date2)
print(date1 == date2)
```

Output:

```
True
False
False
```



This is useful for filtering data.

For example:

</> Python



▶ Run

```
if order_date >= start_date:
    print("Keep order")
```

19. datetime.now() vs datetime.today()

Both give the current local date and time.

```
</> Python

from datetime import datetime

print(datetime.now())
print(datetime.today())
```



▶ Run

Usually you'll see essentially the same result.

I recommend using:

```
</> Python

datetime.now()
```



▶ Run

because it is clearer and commonly used.

20. UTC time

In data engineering, **UTC** is very important.

You can get the current UTC time:

```
</> Python

from datetime import datetime, timezone

now_utc = datetime.now(timezone.utc)

print(now_utc)
```



▶ Run

Example:

```
2026-08-29 17:19:32.452781+00:00
```



Notice:

```
+00:00
```



That means UTC.

21. Naive vs aware datetime

This is an important concept.

Naive datetime

</> Python



Run

```
from datetime import datetime

dt = datetime.now()

print(dt)
```

It doesn't contain timezone information.

Timezone-aware datetime

</> Python



Run

```
from datetime import datetime, timezone

dt = datetime.now(timezone.utc)

print(dt)
```

It contains timezone information.

For distributed systems such as:

Kafka



Spark



HDFS



Data Warehouse



timezone-aware timestamps can prevent many problems.

22. Creating a specific datetime

You can manually create one:

</> Python



Run


```
from datetime import datetime

dt = datetime(
    2026,
    8,
    29,
    20,
    30,
    15
)

print(dt)
```

Output:

2026-08-29 20:30:15



The structure is:

</> Python



▶ Run

```
datetime(
    year,
    month,
    day,
    hour,
    minute,
    second
)
```

23. Creating JSON with datetime

This connects directly to what you're learning.

Suppose you want to generate an order:

</> Python



▶ Run

```
from datetime import datetime
import json

order = {
    "order_id": 1001,
    "product": "Laptop",
    "price": 1000,
    "quantity": 2,
    "event_time": datetime.now().strftime("%Y-%m-%d %H:%M:%S")
}

print(json.dumps(order, indent=4))
```

Output:

```
</> JSON

{
  "order_id": 1001,
  "product": "Laptop",
  "price": 1000,
  "quantity": 2,
  "event_time": "2026-08-29 20:19:32"
}
```

Why use `strftime()` ?

Because Python's `datetime` object isn't directly JSON serializable.

This won't work directly:

```
</> Python

order = {
    "event_time": datetime.now()
}

json.dumps(order)
```

You'll get an error similar to:

```
TypeError: Object of type datetime is not JSON serializable
```

So convert it to a string:

```
</> Python
```

```
datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

24. Generate files based on time

This is particularly useful for your current practice.

Python



Run

```
from datetime import datetime
import json

now = datetime.now()

filename = now.strftime("orders_%Y%m%d_%H%M%S.json")

order = {
    "order_id": 1001,
    "price": 500,
    "quantity": 2,
    "event_time": now.strftime("%Y-%m-%d %H:%M:%S")
}

with open(filename, "w") as file:
    json.dump(order, file, indent=4)

print("Created:", filename)
```

You might get:

```
orders_20260829_201932.json
```



This is a very common pattern for generating test data.

25. Create a file every 10 seconds

You can combine:

- datetime
- json
- time
- file handling

Example:

</> Python



Run

```
import json
import time
from datetime import datetime

while True:

    now = datetime.now()

    filename = now.strftime("orders_%Y%m%d_%H%M%S.json")

    order = {
        "order_id": 1001,
        "price": 500,
        "quantity": 2,
        "event_time": now.strftime("%Y-%m-%d %H:%M:%S")
    }

    with open(filename, "w") as file:
        json.dump(order, file, indent=4)

    print("Created:", filename)

    time.sleep(10)
```

The flow is:

```
Get current time
  ↓
Create filename
  ↓
Create JSON
  ↓
Save file
  ↓
Wait 10 seconds
  ↓
Repeat
```



This is very useful for practicing **Spark Structured Streaming file sources**.

26. Important distinction: datetime vs time

There are two different things:

```
</> Python  
  
import time
```

[Run](#)

and:

```
</> Python  
  
from datetime import datetime
```

[Run](#)

They are not the same.

`time`

The `time` module is commonly used for things like:

```
</> Python  
  
time.sleep(10)
```

[Run](#)

Meaning:

Stop execution for 10 seconds.

`datetime`

Used for actual dates and timestamps:

```
</> Python  
  
datetime.now()
```

[Run](#)

Meaning:

What date/time is it?

So:

```
</> Python
```

[Run](#)

```
import time
from datetime import datetime

print(datetime.now())

time.sleep(5)

print(datetime.now())
```

The program waits 5 seconds between the two timestamps.

27. Very important functions to learn

For your data engineering goals, I recommend mastering these first:

</> Python



▶ Run

```
datetime.now()
```

</> Python



▶ Run

```
datetime.today()
```

</> Python



▶ Run

```
date.today()
```

</> Python



▶ Run

```
datetime(...)
```

</> Python



▶ Run

```
strptime(...)
```

</> Python



▶ Run

```
strptime(...)
```

</> Python



▶ Run

```
timedelta(...)
```

</> Python

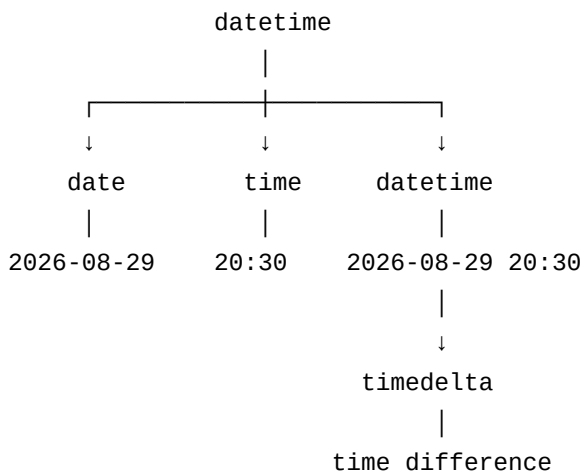


Run

```
datetime.now(timezone.utc)
```

28. The complete mental model

Remember this:



And remember these two conversions:

```
datetime --strftime--> string
datetime <--strptime-- string
```



For example:

</> Python



Run

```
dt = datetime.now()

text = dt.strftime("%Y-%m-%d %H:%M:%S")
```

Then:

</> Python



Run

```
dt2 = datetime.strptime(
    text,
    "%Y-%m-%d %H:%M:%S"
)
```

29. Data engineering example

Imagine a streaming system:

```
Python
  |
  | generates JSON
  ↓
JSON files
  |
  ↓
HDFS
  |
  ↓
Spark Structured Streaming
  |
  ↓
Transformations
  |
  ↓
Parquet
```



Your Python producer could create:

```
</> JSON

{
  "order_id": 101,
  "product": "Laptop",
  "price": 1200,
  "quantity": 2,
  "event_time": "2026-08-29 20:20:00"
}
```



Then Spark can interpret:

```
event_time
  ↓
TimestampType
  ↓
event-time processing
  ↓
windows
  ↓
watermarks
```



For example, later you'll use Spark functions such as:

</> Python



▶ Run

```
from pyspark.sql.functions import col, to_timestamp

df = df.withColumn(
    "event_time",
    to_timestamp(col("event_time"))
)
```

And eventually:

</> Python



▶ Run

```
.groupBy(
    window("event_time", "10 minutes")
)
```

So learning Python `datetime` now directly prepares you for **event-time processing in Spark Structured Streaming**.



ChatGPT can make mistakes. Check important info.